I. FSM

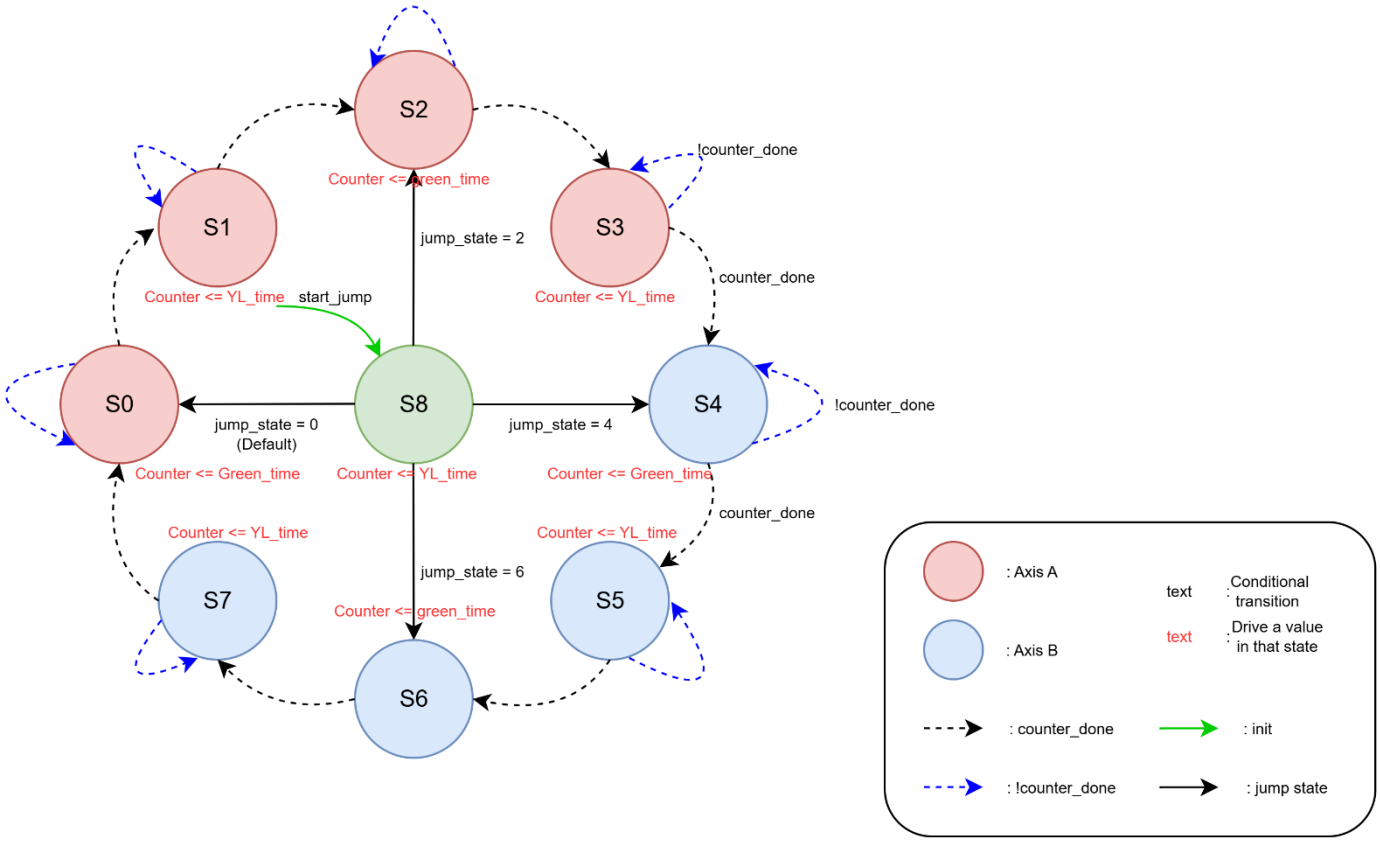
I. Traffic light state combinations

State	Axis A- South			Axis A - North			Axis B - East			Axis B - West		
	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)
S0: A_STRAIGHT	Green	Red	Green	Green	Red	Green	Red	Red	Red	Red	Red	Red
S1: A_S_YELLOW	Yellow	Red	Red	Yellow	Red	Red	Red	Red	Red	Red	Red	Red
S2: A_LEFT	Red	Green	Red	Red	Green	Red	Red	Red	Red	Red	Red	Red
S3: A_L_YELLOW	Red	Yellow	Red	Red	Yellow	Red	Red	Red	Red	Red	Red	Red
S4: B_STRAIGHT	Red	Red	Red	Red	Red	Red	Green	Red	Green	Green	Red	Green
S5: B_S_YELLOW	Red	Red	Red	Red	Red	Red	Yellow	Red	Red	Yellow	Red	Red
S6: B_LEFT	Red	Red	Red	Red	Red	Red	Red	Green	Red	Red	Green	Red
S7: B_L_YELLOW	Red	Red	Red	Red	Red	Red	Red	Yellow	Red	Red	Yellow	Red

Simplify							
State	Axis A			Axis B			Note
	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)	
S0: A_STRAIGHT	Green	Red	Green	Red			
S1: A_S_YELLOW	Yellow	Red	Red				Add all red after yellow
S2: A_LEFT	Red	Green	Red				
S3: A_L_YELLOW	Red	Yellow	Red				Add all red after yellow
S4: B_STRAIGHT	Red			Green	Red	Green	
S5: B_S_YELLOW				Yellow	Red	Red	Add all red after yellow
S6: B_LEFT				Red	Green	Red	
S7: B_L_YELLOW				Red	Yellow	Red	Add all red after yellow
S8: Idel for Jump	If green, change to yellow; otherwise, keep the previous state, using register						

Actually, after the yellow light, there's a state where all directions are red. This gives the traffic caught in the yellow light enough time to flush out of the junction

II. FSM



- Based on the state combination table above, we can derive the state transition diagram. States S0 through S7 represent the normal operational sequence, while state S8 is designed to force a transition to any specific state (conceptually similar to a manual override by a traffic police officer to regulate traffic)

- S8 serves as an idle state that waits for the start_jump force signal. This signal is asserted when any 1 of the 12 manual override buttons (3 buttons per direction) is pressed to jump for green light

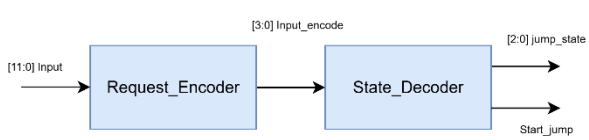
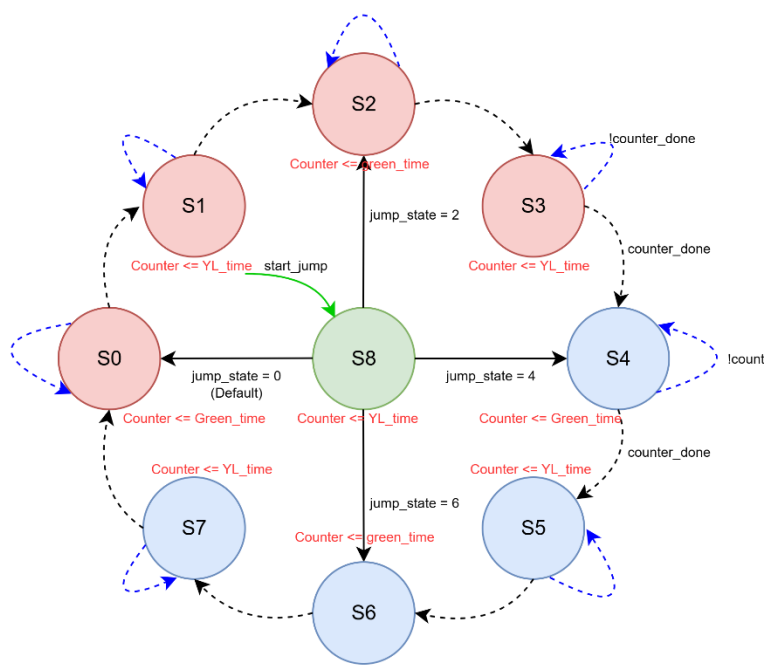
-When enter S8 will be yellow light

III. Control signal

A. Bottons request

Resquest	Axis A- South			Axis A - North			Axis B - East			Axis B - West			ENCODER	
	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)	Go straight (S)	Go left (F)	Pedestrian(P)		
1 is resquest	0												0	0000
												1	1	0001
											1		2	0010
										1			3	0011
									1				4	0100
								1					5	0101
							1						6	0110
						1							7	0111
					1								8	1000
				1									9	1001
			1										10	1010
		1											11	1011
	1												12	1100

-There are 12 input buttons in total. The system enforces mutual exclusion, meaning only one button can be activated at a time on a first-come, first-served basis, so i encode 12 to 4 bit for easy control.

ENCODER	Request		jump_state	Decoder	Start_jump
0000	None			0	0
0001	Axis B - West	Pedestrian(P)	S4: B_STRAIGHT	4	1
0010		Go left (F)	S6: B_LEFT	6	
0011		Go straight (S)	S4: B_STRAIGHT	4	
0100	Axis B - East	Pedestrian(P)	S4: B_STRAIGHT		
0101		Go left (F)	S6: B_LEFT	6	
0110		Go straight (S)	S4: B_STRAIGHT	4	
0111	Axis A - North	Pedestrian(P)	S0: A_STRAIGHT	0	
1000		Go left (F)	S2: A_LEFT	2	
1001		Go straight (S)	S0: A_STRAIGHT	0	
1010	Axis A- South	Pedestrian(P)	S0: A_STRAIGHT		
1011		Go left (F)	S2: A_LEFT	2	
1100		Go straight (S)	S0: A_STRAIGHT	0	

From the button table, we can decode the desired traffic light state. If a button is pressed indicated by a non-zero 4-bit value so we enable the jump by asserting start_jump to 1. Conversely, a value of 0000 indicates no button press, meaning the transition into state S8 is not enabled